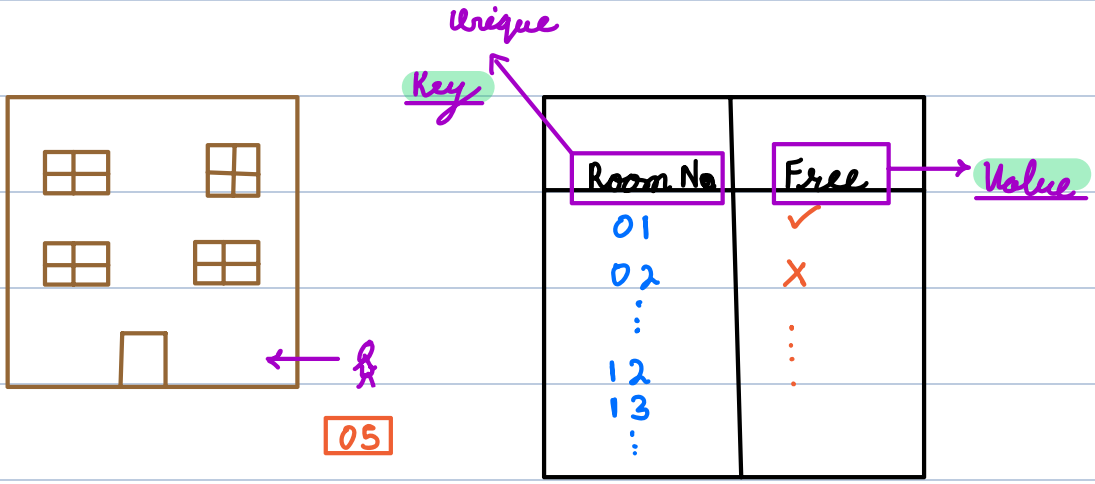
Agenda:

1. Introduction to HashMap & it's functions
2. Given queries, find frequency of an element
3. Introduction to HashSet & it's functions
4. No. of Distinct Elements
5. Iterate over HashMap & HashSet
6. Find any pair (i, j) s.t. A[i] = A[j] & abs(i - j) is minimum

Hotel



Hash Table / Hash Map

Room → capacity, AC, Balcony, ...

Key collection of values

Which of the following HashMap will you use to store:

1. Population of every country

🇮🇳 India → 145

🇺🇸 USA → 65

🇷🇺 Russia → 14

🇨🇳 China → 147

HashMap < String , Long >

2. Name of all the states of every country

🇮🇳 India → Andhra Pradesh, Arunachal Pradesh, U.P, M.P, Karnataka,....

🇺🇸 USA → New York, Washington, Texas,...

🇷🇺 Russia → Moscow, Kazan, Samara,...

HashMap < String , List < String >>

Name, age

3. Population of each state in each country

🇮🇳 **India**

Karnataka → 50
Maharashtra → 72
Himachal Pradesh → 36

🇺🇸 **USA**

Texas → 18
Florida → 16
Washington → 32

🇷🇺 **Russia**

Kazan → 3
Samara → 5
Moscow → 7

HashMap<String,
HashMap<String, Integer>>

Here are some essential operations on HashMap:

Method	Description	Example
`put(K key, V value)`	Inserts a key-value pair.	`map.put("John", 25);`
`get(Object key)`	Retrieves a value using a key.	`map.get("John");`
`remove(Object key)`	Removes a key-value pair.	`map.remove("John");`
`containsKey(Object key)`	Checks if a key exists.	`map.containsKey("John");`
`containsValue(Object value)`	Checks if a value exists.	`map.containsValue(25);`
`size()`	Returns the number of key-value pairs.	`map.size();`
`isEmpty()`	Checks if the map is empty.	`map.isEmpty();`

Q1: Given **N** elements and **Q** queries, find the **frequency** of the elements provided in a query.

Example

N = 10

2	6	3	8	2	8	2	3	8	10	6

Q = 4

2	8	3	5

Solution

Element	Frequency
2	3
8	3
3	2
5	0

Frequency array →

Range <= 10^7

$\boxed{-10^9 <= A[i] <= 10^9}$

Key → A[i]

Value → frequency of A[i]

```
∀i → F[i] = 0

for i → 0 to (N-1) {
|     F[A[i]] ++
}
```

Pseudo-code

```java
public class DSA101 {
    public static void main(String[] args) {
        HashMap<Integer, Integer> freq = new HashMap<>();

        int a[] = {2, 6, 3, 8, 2, 8, 2, 3, 8, 10, 6};
        int n = a.length;
        for(int i = 0; i < n; i++) {
            int f = freq.getOrDefault(a[i], 0);
            freq.put(a[i], f + 1);
        }

        int query[] = {2, 8, 3, 5};
        for(int x : query) {
            if(freq.containsKey(x))
                System.out.println(x +" -> "+ freq.get(x));
            else
                System.out.println(x +" -> 0");
        }
    }
}
// TC = O(N + Q)
// SC = O(N) - Max size of HashMap
```

HashSet

- HashMap without value
- Only store unique keys

```
HashSet<Integer> hs = new HashSet<>();
```

datatype — variable name

Commonly Used Methods in HashSet

`add(E e)`	Adds the element to the set if not already present.
`addAll(Collection<? extends E> c)`	Adds all elements from another collection.
`remove(Object o)`	Removes the specified element.
`contains(Object o)`	Checks if the set contains the element.
`isEmpty()`	Returns true if the set is empty.
`size()`	Returns the number of elements in the set.
`clear()`	Removes all elements from the set.

Q2: Given an array of N elements, find the count of **distinct** elements.

Input:

N = 5

3	5	6	5	4

Output:

ans = 4

Input:

N = 3

3	3	3

Output:

ans = 1

Input:

N = 5

1	1	1	2	2

Output:

ans = 2

```java
public static void main(String[] args) {
    int a[] = {3, 5, 6, 5, 4};
    HashSet<Integer> hs = new HashSet<>();
    for(int x : a) {
        hs.add(x);
    }
    System.out.println(hs.size());
}
```

TC = O(N) SC = O(N)

Iterate over HashMap & HashSet

ITERATING OVER A HASHMAP

A HashMap stores data as **Key → Value**. So iteration can be done in **3 ways**:

1. Iterate over Keys

Useful when you only need to process key information.

```
for (String key : map.keySet()) {
    System.out.println(key);
}
```

2. Iterate over Values

Useful when keys are irrelevant.

```
for (Integer value : map.values()) {
    System.out.println(value);
}
```

3. Iterate over Key–Value Pairs (Most Common)

```
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

ITERATING OVER A HASHSET

HashSet only stores **unique keys**, so iteration is simpler.

Iterate Over All Elements

```
for (Integer x : set) {
    System.out.println(x);
}
```

Key Observations & Tips

1. **Order Is Not Guaranteed**
   HashMap/HashSet do **not** preserve order.
   Every iteration gives data in **any arbitrary order**.
2. **Use Entry Set for Efficiency (Java)**
   Iterating via `entrySet()` is **faster** than keySet → get() calls.
3. **Avoid Modifying While Iterating**
   Removing/adding while iterating may cause errors or undefined behavior.

Q3: You are given an array **A[ ]** of size **N**. A **special pair** is defined as:

- Two different indices **(i, j)** such that **A[i] == A[j]**
- The distance between them is: **|i - j|**

Your task:

👉 Among all special pairs, find the **minimum distance**.

👉 If no such pair exists, return **-1**.

Example: **A** = [7, 1, 3, 7, 1, 7] ans = **2**

Bruteforce → ∀i, j check if A[i] == A[j] & i != j.

Store min |i - j|. TC = O(N²) SC = O(1)

A = [7 1 3 7 1 7]

HashMap

Key → Value

A[i] → prev. Index

7 → ~~0~~ ~~3~~ 5

1 → ~~1~~ 4

3 → 2

|3 - 0| = 3

|4 - 1| = 3

|5 - 3| = 2 (Ans)

1 <= ans <= N-1

```java
public class Solution {
    public int solve(int[] A) {
        if (A.length == 1) {
            return -1;
        }
        // stores <Value, Index> pair
        HashMap < Integer, Integer > map = new HashMap < Integer, Integer > ();
        int result = A.length;
        for (int i = 0; i < A.length; i++) {
            // checks if A[i] has occurred previously
            if (map.containsKey(A[i])) {
                result = Math.min(result, i - map.get(A[i]));
            }
            map.put(A[i], i);
        }
        return (result == A.length) ? -1 : result;
    }
}
```

TC = O(N) SC = O(N)

What is the size of the set and one possible output order?

```java
import java.util.*;

public class Test2 {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();

        set.add(10);
        set.add(20);
        set.add(10);
        set.add(30);
        set.add(20);

        System.out.println("size = " + set.size());
        for (int x : set) System.out.print(x + " ");
    }
}
```

→ 3

→ random